Learning Agents: Regression and Classification

Lecture 26

CS 4880 Kirk Duran

Today: From Route Logs to a Useful Predictor

1. Turn an agent's experience into labeled examples and a prediction task.
2. Define a model family and loss; fit and interpret a least-squares line.
3. Explain how optimization and feature transformations affect the fit.
4. Compare regression and classification: outputs, losses, and held-out evaluation.

Key idea

Our running question: can past deliveries improve the travel-time estimates used by a planner?

Detailed derivations, regularization, and a code example are optional reference material.

An Agent Should Improve with Experience

Our robot can plan routes, but its travel-time estimates may be inaccurate.

- A fixed rule might assume 2 minutes per block.
- Actual time varies with congestion and surface conditions.
- Completed deliveries pair route inputs with actual times.
- Use those examples to revise future predictions.



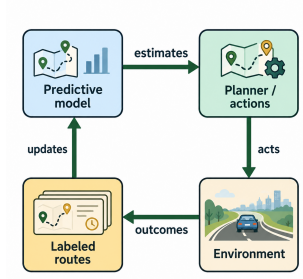
Key idea

A learning agent improves a model from experience to support future decisions.

Where Learning Fits Inside an Agent

- **Performance element:** selects actions using current predictions.
- **Learning element:** updates a predictive model from examples.
- **Observed feedback:** records actual travel times after actions.

This is a simplified feedback loop. Prediction supports planning; it is not itself an action policy.



Key idea

A learning element can predict a number or a category from known inputs.

From Hand-Coded Relationships to Learned Functions

A traditional model specifies a relationship in advance:

$$\widehat{t} = 2 \cdot \text{distance}.$$

A learned model chooses parameters from data:

$$\widehat{t} = f_{\boldsymbol{\theta}}(\mathbf{x}), \qquad \mathbf{x} = \big(\text{distance}, \text{traffic}, \ldots\big).$$

Specified by the designer

Relevant input features, model family, and learning objective.

Estimated from experience

Numerical parameters $\boldsymbol{\theta}$ that minimize the chosen training objective.

Key idea

Learning estimates a model from examples; it does not remove modeling assumptions.

Learning Needs a Measurable Target

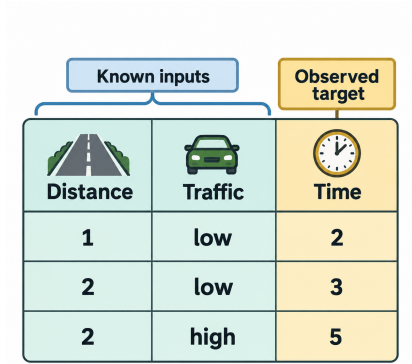
Each recorded delivery creates a training example:

$$(\mathbf{x}_i, y_i),$$

where

- $\mathbf{x}_i$ contains known route features;
- y_i is the observed travel time;
- $\widehat{y}_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$ is the prediction.

Inputs must be available *before* the outcome occurs. The goal is to predict y for *new* routes, not merely reproduce old records.



Examples, Models, and New Observations

Consider an email service that learns how to process incoming messages.

Training example Features of one historical message, paired with its observed target: $(\mathbf{x}_i, y_i)$.

Learned model A fitted function $f_{\widehat{\boldsymbol{\theta}}}$, shared across messages; its parameters were estimated from many examples.

New observation Features $\mathbf{x}_{\text{new}}$ available when a message arrives; its target is not yet known.

Key idea

Training uses observed targets to adjust parameters. Inference applies the fitted model to a new input.

What Makes Learning "Supervised"?

Definition

Supervised learning uses examples $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with observed targets y_i to learn a mapping from inputs to outputs.

Regression

Predict a real-valued outcome: travel time, battery usage, or a measured distance.

$$f : \mathbb{R}^d \to \mathbb{R}.$$

Classification

Predict a discrete category: obstacle versus clear path, or safe versus unsafe.

$$f : \mathbb{R}^d \to \{1, \ldots, K\}.$$

Key idea

n: labeled examples; d: input features; K: categories. We develop **regression** first, then a **binary classifier**.

The Same Inputs Can Support Different Targets

Email features could include message length, link count, a known-sender indicator, and attachment size.

Regression task

Predict handling time in seconds.

Target: a measured duration.
Output: a numerical estimate.
Loss: squared or absolute error.

Classification task

Predict spam versus legitimate.

Target: 1 for spam, 0 otherwise.
Output: a score, then a class.
Loss: a classification objective.

Key idea

The target and loss define the task. A later moderator's verdict can be a label, but cannot be an input available on arrival.

Data Are Samples, Not the Whole World

In statistical learning, assume examples arise from an unknown distribution:

$$(\mathbf{X}, Y) \sim P_{X,Y}, \qquad \mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^{n}.$$

A common starting assumption is independent, identically distributed (i.i.d.) samples.

- Training examples reveal only part of the relationship.
- Measurement noise and unobserved variables affect outcomes.
- Repeated measurements of the same route may be correlated; a random split can then be misleading.

Key idea

Generalization claims depend on how future examples relate to the training sample. A new campus or weather regime may shift that distribution.

Hypothesis Classes Express Inductive Bias

We choose a family of candidate predictors:

$$\mathcal{H} = \{f_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \Theta\}.$$

Examples of curve-fitting models include

- straight lines: $f(x) = w_0 + w_1 x$;
- polynomial curves: $f(x) = \sum_{j=0}^{p} w_j x^j$;
- linear combinations of chosen basis functions: $f(x) = \sum_j w_j \phi_j(x)$.

Inductive bias

$\mathcal{H}$ is the hypothesis class; Θ is its allowed parameter set. These restrictions guide generalization but can exclude useful relationships.

A Loss Function Defines What “Good” Means

For one prediction, define a cost $\ell(\widehat{y}, y)$.

$$\ell_{\mathrm{sq}}(\widehat{y}, y) = (y - \widehat{y})^2, \qquad \ell_{\mathrm{abs}}(\widehat{y}, y) = |y - \widehat{y}|.$$

Squared loss

Differentiable; penalizes a large miss disproportionately more.

Absolute loss

Measures error in original units; less sensitive to extreme residuals.

Key idea

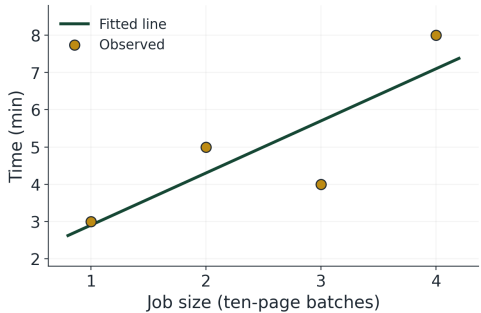
Fitting a model means optimizing a precisely stated objective, not merely drawing a visually appealing line.

First Model: A Straight Line

A separate example: predict a print job's completion time from its size.

$$\widehat{y} = w_0 + w_1 x.$$

- x: number of ten-page batches;
- w_0: predicted setup time;
- w_1: additional minutes per batch.



Key idea

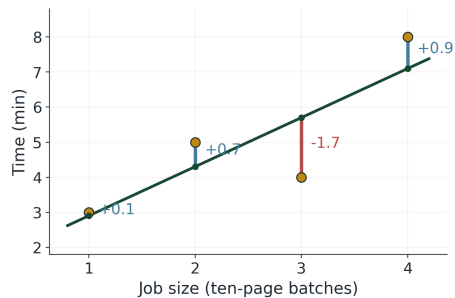
Linear regression is linear in its fitted coefficients w_0, w_1.

Residuals Measure the Error of Each Prediction

For each observed pair (x_i, y_i), define

$$e_i = y_i - \widehat{y}_i = y_i - (w_0 + w_1 x_i).$$

- Positive residual: the model underpredicted.
- Negative residual: the model overpredicted.
- Residuals retain their sign and units.
- The opposite convention $r_i = \widehat{y}_i - y_i = -e_i$ gives identical squared and absolute errors.



Ordinary Least Squares

Choose coefficients to minimize the sum of squared residuals:

$$S(w_0, w_1) = \sum_{i=1}^{n} \big[y_i - (w_0 + w_1 x_i)\big]^2.$$

Mean squared error is $\mathrm{MSE} = S/n$, with the same minimizing coefficients.

Geometry of the objective

In w_0, w_1, the objective is a convex quadratic. A stationary point is a global minimum; distinct input values make the fitted line unique.

Key idea

Curve fitting has become an optimization problem: find the parameters that minimize aggregate error.

Measuring Numerical Prediction Error

For n examples in the set being evaluated:

$$\mathrm{MSE} = \frac{1}{n}\sum_i (y_i - \widehat{y}_i)^2, \qquad \mathrm{RMSE} = \sqrt{\mathrm{MSE}},$$

$$\mathrm{MAE} = \frac{1}{n}\sum_i |y_i - \widehat{y}_i|.$$

- If the target is in minutes, MSE is in minutes squared; RMSE and MAE are in minutes.
- MSE and RMSE emphasize large errors more than MAE.
- Compare models on the *same examples* using the *same metric*.

Key idea

Parameters define the predictor. A loss evaluates predictions and supplies the quantity minimized during fitting.

The Least-Squares Coefficients

For a line with an intercept, the minimizing coefficients are

$$\boxed{w_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}}, \qquad \boxed{w_0 = \overline{y} - w_1 \overline{x}}.$$

- $\overline{x} = \frac{1}{n}\sum_i x_i$ and $\overline{y} = \frac{1}{n}\sum_i y_i$ are sample means.
- The numerator measures how inputs and targets vary together.
- The denominator measures input spread; it must be nonzero.
- The fitted line passes through $(\overline{x}, \overline{y})$.

Key idea

Solve the optimization problem by setting both partial derivatives to zero. The derivation is in the reference slides; now apply the result.

Worked Example: Four Print Jobs

Job i	1	2	3	4
Size (ten-page batches) x_i	1	2	3	4
Time (min) y_i	3	5	4	8

$$\overline{x} = 2.5, \qquad \overline{y} = 5,$$

$$\sum_i (x_i - \overline{x})(y_i - \overline{y}) = 7, \qquad \sum_i (x_i - \overline{x})^2 = 5.$$

Key idea

Job size does not determine every observed time. The synthetic data include variation the single input cannot explain.

Worked Example: Fit and Predict

$$w_1 = \frac{7}{5} = 1.4, \qquad w_0 = 5 - (1.4)(2.5) = 1.5,$$

$$\boxed{\widehat{y} = 1.5 + 1.4x}.$$

At $x = 5$ batches, $\widehat{y} = 8.5$ min. This is **extrapolation**: the observed sizes span only 1–4 batches.

Observed training times	Fitted training times
3, 5, 4, 8	2.9, 4.3, 5.7, 7.1

Key idea

The fitted line need not pass through the observations. A prediction beyond the observed range needs additional evidence.

Numerical Check: How Good Was Our Line?

For the four print jobs, observed-minus-predicted residuals are

$$\mathbf{e} = (0.1,\ 0.7,\ -1.7,\ 0.9)\ \text{min}.$$

$$\text{MSE} = \frac{0.01 + 0.49 + 2.89 + 0.81}{4} = 1.05\ \text{min}^2,$$

$$\text{RMSE} = \sqrt{1.05} \approx 1.025\ \text{min}, \qquad \text{MAE} = \frac{3.4}{4} = 0.85\ \text{min}.$$

Interpretation

The third observation dominates squared error. These are *training-fit* measures; they do not estimate accuracy on unseen jobs.

Multiple Input Features: Still Linear Regression

Route time might depend on several measurements:

$$\widehat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d.$$

With a leading constant feature $x_0 = 1$, write

$$\widehat{y} = \mathbf{w}^{\mathsf{T}} \widetilde{\mathbf{x}}, \qquad \widetilde{\mathbf{x}} = (1, x_1, \ldots, x_d)^{\mathsf{T}}.$$

- One coefficient per feature, plus an intercept.
- Each slope describes an additive change in the prediction with other features held fixed.
- Coefficients alone do not establish causal effects.

Key idea

"Linear" refers to dependence on the coefficients, not necessarily to one input variable.

From Residuals to a Parameter Update

Use the prediction-minus-observation convention $r_i = w_0 + w_1 x_i - y_i$.

$$L = \frac{1}{n}\sum_i r_i^2, \qquad g_0 = \frac{\partial L}{\partial w_0} = \frac{2}{n}\sum_i r_i, \qquad g_1 = \frac{\partial L}{\partial w_1} = \frac{2}{n}\sum_i x_i r_i.$$

- g_0: changing the intercept shifts every prediction equally.
- g_1: the slope change is weighted by each input x_i.
- The gradient points toward locally increasing loss; subtract it to descend.

Key idea

Compute both gradients from the *same current parameters*, then update $w_0 \leftarrow w_0 - \eta g_0$ and $w_1 \leftarrow w_1 - \eta g_1$ simultaneously.

One Gradient Step on the Print Jobs

Start from $(w_0, w_1) = (0.5, 1)$ and learning rate $\eta = 0.05$.

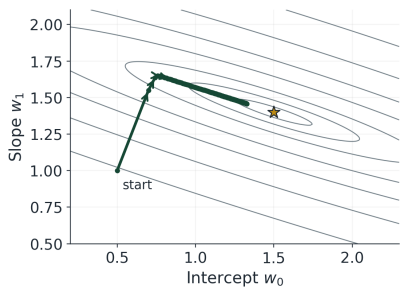
All residuals use the old line

Predictions: $1.5,\ 2.5,\ 3.5,\ 4.5.$
$\widehat{y} - y$: $-1.5,\ -2.5,\ -0.5,\ -3.5.$

$$g_0 = -4, \qquad g_1 = -11.$$

New parameters: $(0.7, 1.55)$.
MSE: $5.25 \longrightarrow 1.25875$.

$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \nabla L(\mathbf{w}^{(k)})$; k counts updates.



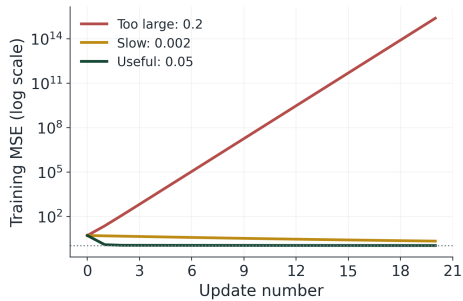
Optimization Has Practical Failure Modes

Step size too large

Updates may overshoot the optimum or diverge.

Step size too small

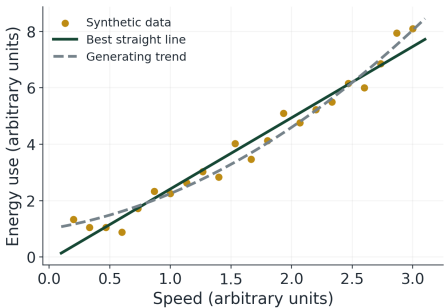
Optimization progresses slowly.



- Standardize inputs with very different scales; estimate scaling from training data only.
- Track the training objective during optimization.
- Distinguish failure to *optimize* from failure to *generalize*.

What If the Relationship Is Not a Straight Line?

A robot's energy use might increase nonlinearly with speed.



Linear-model mismatch

No choice of w_0, w_1 can bend a straight line to follow curved data.

Next step

Transform the inputs; keep the model linear in its parameters.

A poor fit may reflect an inadequate **hypothesis class**, not an inadequate optimizer.

Polynomial Regression Is Linear in the Coefficients

Transform one input into multiple basis functions:

$$\boldsymbol{\phi}(x) = (1, x, x^2, \ldots, x^p)^{\mathsf{T}}.$$

Then fit

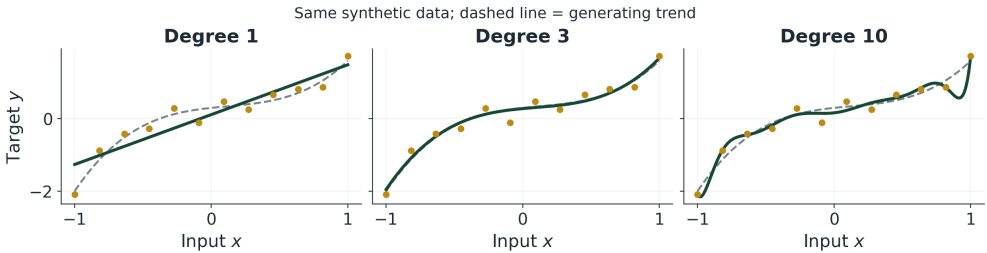
$$\widehat{y} = \mathbf{w}^{\mathsf{T}}\boldsymbol{\phi}(x) = w_0 + w_1 x + \cdots + w_p x^p.$$

- The curve is nonlinear in x but remains linear in $\mathbf{w}$.
- Least squares still applies using transformed feature columns.
- Degree p: 0 is constant, 1 a line, 2 quadratic; larger p adds flexibility.

Key idea

For $p = 0$, least squares predicts the training mean $\widehat{y} = \overline{y}_{\mathrm{tr}}$. This is a simple baseline that ignores the input.

Model Capacity: Degree One, Three, and Ten



Underfitting

Limited flexibility; may miss important structure.

Useful structure

May capture the stable underlying pattern.

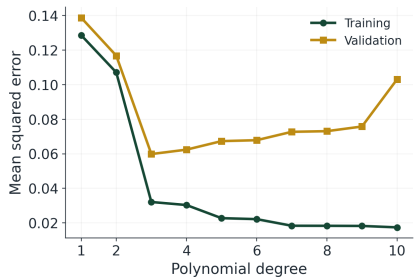
Overfitting risk

Can chase noise and behave erratically between points.

Training Error and Generalization Error Differ

We fit parameters using training examples, but judge usefulness on unseen examples.

- For nested model families, the best achievable training error cannot increase.
- Validation error need not decrease with complexity; its shape is data-dependent.
- The plot uses the same synthetic training sample for each polynomial degree.

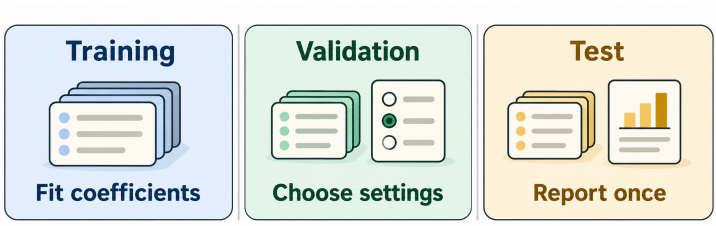


Key idea

The target is low error *on the distribution of future examples*, not the smallest possible training error.

Train, Validation, and Test Have Different Jobs

- **Parameters** are fitted: coefficients $\mathbf{w}$.
- **Hyperparameters** are selected: degree p, penalty λ.



Key idea

Repeatedly choosing models based on test results makes the test set part of training and biases the reported result.

A Useful Evaluation Must Resemble Deployment

Prevent information leakage

- Use inputs available at prediction time: not the delivery's eventual arrival timestamp.
- Split first; fit scaling, imputation, and feature selection on training data only.
- Apply the same fitted transformations to validation and test data.

Match the intended prediction task

- Future deliveries: consider a chronological split.
- Unseen routes: hold out entire route groups.
- New conditions can cause distribution shift even with a careful split.

Why More Flexibility Does Not Always Help

- **Bias:** a restricted family systematically misses the conditional trend.
- **Variance:** a fitted curve changes substantially when the training sample changes.
- **Irreducible noise:** unobserved factors still make outcomes vary at a fixed input.

Connect this to the polynomial fits

A line may miss curvature. A high-degree fit can follow this sample's noise. Neither a lower training error nor a higher degree proves better future performance.

Key idea

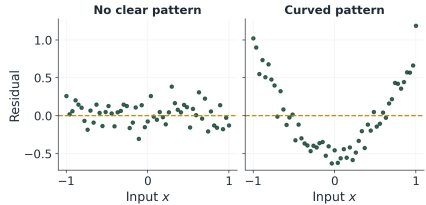
More training data can reduce variance; greater noise makes chasing sample details more hazardous. Select flexibility using held-out data.

The formal squared-error decomposition is in the reference slides.

Residual Diagnostics Reveal Model Weaknesses

Plot residuals $e_i = y_i - \widehat{y}_i$ against fitted values or inputs.

- Curved pattern: missing systematic nonlinear structure.
- Increasing spread: non-constant noise variance.
- Extreme residuals: possible outliers or exceptional conditions.
- Time dependence or abrupt shifts: structure not captured by one smooth curve.



Key idea

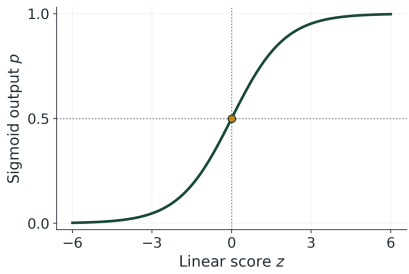
The magnitude of error tells us *how much* the fit misses; its structure can suggest *why*.

A Linear Score Can Become a Binary Classifier

Return to email: $y = 1$ means spam; $y = 0$ means legitimate.

$$z = w_0 + \sum_{j=1}^{d} w_j x_j, \qquad p_{\mathbf{w}}(\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

- z: unrestricted real-valued score.
- p: model estimate of $P(Y = 1 \mid \mathbf{x})$, between 0 and 1.
- **Logistic regression** is a classification model despite its name.



Classification Changes the Training Objective

For a binary label $y \in \{0, 1\}$ and predicted spam probability p:

$$\ell(p, y) = -\big[y \log p + (1 - y) \log(1 - p)\big].$$

- If $y = 1$, loss is $-\log p$; if $y = 0$, it is $-\log(1 - p)$.
- For a spam message: $p = 0.8$ gives loss 0.223; $p = 0.2$ gives 1.609.
- Fit weights by minimizing average **binary cross-entropy** (log loss), often with a regularization penalty.

Key idea

The same learning recipe remains: choose a model, define a loss, fit parameters, and evaluate on unseen examples. Here the model estimates a class probability rather than a numerical duration.

Logs are natural logarithms. A confidently wrong prediction incurs a large loss.

A Threshold Turns a Probability into a Label

Predict spam when $p \geq \tau$, otherwise legitimate; τ is the threshold. For example, $p = 0.7$ is labeled spam at $\tau = 0.5$, but legitimate at $\tau = 0.8$. Suppose the matrix below was measured at $\tau = 0.5$:

100 hypothetical messages	**Predicted legitimate**	**Predicted spam**
Actual legitimate	64 true negatives	6 false positives
Actual spam	9 false negatives	21 true positives

- A **confusion matrix** counts actual classes by row, predicted classes by column. Here the positive class is spam.
- A false positive hides useful mail; a false negative lets spam through. Their operational costs can differ.
- Lowering τ can trade false negatives for false positives. Choose settings with validation data.

A Bounded Score Is Not Automatically Calibrated

The sigmoid constrains a score's range. It does not prove the score matches real frequencies.

What calibration means

Among many new messages assigned scores near 0.8, approximately 80% should actually be spam if those probabilities are well calibrated.

- **Classification:** which label does a threshold assign?
- **Calibration:** do the probability estimates agree with observed frequencies?
- A useful ranking or a good confusion matrix at one threshold does not establish calibration.

Key idea

Inspect held-out outcomes. A numerical estimate supports a decision, but its range alone is not evidence of reliability.

Check Your Understanding

In class

1. A flexible curve lowers training error. What evidence is still missing?
2. A spam filter produces $p = 0.7$. Is that a label, a guarantee, or a model estimate?
3. If useful messages are being hidden, what evaluation evidence would help choose a different threshold?

Key idea

Separate the fitted model, the loss used to learn it, and the decisions made from its predictions. Judge usefulness on representative unseen data.

Takeaways: Supervised Prediction

- Training examples pair inputs with observed targets; inference uses a fitted model on new inputs.
- Regression predicts numbers; a binary classifier estimates a probability and uses a threshold to assign a label.
- Loss functions define the objective: squared error and cross-entropy reward different predictions.
- Gradient descent updates all parameters from the same current state.
- Curve flexibility helps only when its gains transfer to new examples.
- Use appropriate metrics, baselines, and held-out data; avoid unavailable inputs.

Key idea

Fitting observations is an optimization task. Establishing useful future predictions is an empirical evaluation task.

Optional Reference: Theory and Implementation

Empirical Risk Versus Prediction Risk

The quantity we would like to minimize is expected loss on future examples:

$$R(f) = \mathbb{E}_{(\mathbf{X},Y)\sim P_{X,Y}}\left[\ell(f(\mathbf{X}), Y)\right].$$

Since the data distribution is unknown, training minimizes its sample analogue:

$$\widehat{R}_{\mathcal{D}}(f) = \frac{1}{n}\sum_{i=1}^{n} \ell(f(\mathbf{x}_i), y_i), \qquad \widehat{f} = \arg\min_{f\in\mathcal{H}} \widehat{R}_{\mathcal{D}}(f).$$

Key idea

R is population risk; $\widehat{R}_{\mathcal{D}}$ is training risk; $\widehat{f}$ is a fitted predictor. ERM minimizes the observed average, not a guarantee about future data.

Deriving the Best-Fitting Intercept

At an optimum, differentiate the sum of squares with respect to w_0:

$$\frac{\partial S}{\partial w_0} = -2 \sum_i \big[y_i - (w_0 + w_1 x_i) \big] = 0.$$

Rearranging,

$$\sum_i y_i = n w_0 + w_1 \sum_i x_i, \qquad \boxed{w_0 = \overline{y} - w_1 \overline{x}}.$$

Key idea

The least-squares line with an intercept passes through the sample mean point $(\overline{x}, \overline{y})$.

Deriving the Best-Fitting Slope

Set the derivative with respect to w_1 to zero:

$$\frac{\partial S}{\partial w_1} = -2 \sum_i x_i \big[y_i - (w_0 + w_1 x_i) \big] = 0.$$

Substitute $w_0 = \overline{y} - w_1 \overline{x}$ and simplify:

$$\boxed{w_1 = \frac{\sum_i (x_i - \overline{x})(y_i - \overline{y})}{\sum_i (x_i - \overline{x})^2}}, \qquad w_0 = \overline{y} - w_1 \overline{x}.$$

Condition

The denominator must be nonzero: at least two training inputs must have distinct values.

Least Squares in Matrix Form

Each row of $X \in \mathbb{R}^{n \times (d+1)}$ is $(1, x_{i1}, \ldots, x_{id})$. Here X denotes a *matrix*, not a random input. The vectors $\mathbf{y}, \widehat{\mathbf{y}} \in \mathbb{R}^n$ hold targets and predictions:

$$\widehat{\mathbf{y}} = X\mathbf{w}, \qquad S(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|_2^2.$$

Differentiate and set the gradient to zero:

$$\nabla S(\mathbf{w}) = 2X^{\mathsf{T}}(X\mathbf{w} - \mathbf{y}) = 0.$$
$$X^{\mathsf{T}}X\widehat{\mathbf{w}} = X^{\mathsf{T}}\mathbf{y}.$$

Full-column-rank case

If $X^{\mathsf{T}}X$ is invertible, $\widehat{\mathbf{w}} = (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y}$.

Key idea

Rank deficiency can make coefficients nonunique. Use QR or SVD in software; do not explicitly compute the inverse.

What Squared Loss Is Really Estimating

For a fixed input x and a constant candidate prediction a,

$$\mathbb{E}\big[(Y-a)^2 \mid X=x\big] = \mathrm{Var}(Y \mid X=x) + \big(a - \mathbb{E}[Y \mid X=x]\big)^2.$$

Therefore the optimal squared-error prediction is

$$\boxed{f^*(x) = \mathbb{E}[Y \mid X=x]}.$$

Interpretation

With finite second moments, squared loss targets the conditional mean. A restricted model class may only approximate it; Gaussian noise is not required to fit least squares.

Key idea

An imperfect fit to individual points can still be an excellent estimate of the underlying conditional trend.

Bias, Variance, and Irreducible Noise

With squared error, predictive performance can be understood through

$$\mathbb{E}_{\mathcal{D},Y|x}\left[(Y - \widehat{f}_{\mathcal{D}}(x))^2\right] = \left(\mathbb{E}_{\mathcal{D}}[\widehat{f}_{\mathcal{D}}(x)] - f^*(x)\right)^2 + \mathrm{Var}_{\mathcal{D}}(\widehat{f}_{\mathcal{D}}(x)) + \sigma^2(x),$$

Here $f^*(x) = \mathbb{E}[Y \mid X = x]$; a fresh test outcome is independent of training set $\mathcal{D}$ given x, and $\sigma^2(x) = \mathrm{Var}(Y \mid X = x)$.

- The first term is squared bias: systematic error averaged over training sets.
- The second is variation over training sets, not variation in the inputs.
- **Noise:** uncertainty in Y remains even with the best predictor.

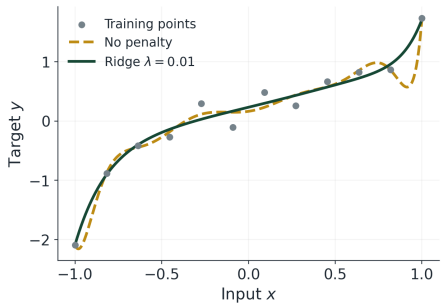
Key idea

Increasing model complexity can reduce bias while increasing sensitivity to sample noise.

Regularization Prefers Simpler Solutions

$$\widehat{\mathbf{w}} = \arg\min_{\mathbf{w}} \left\{ \frac{1}{n} \sum_i [y_i - f_{\mathbf{w}}(\mathbf{x}_i)]^2 + \lambda \sum_{j=1}^{m} w_j^2 \right\}, \quad \lambda \geq 0.$$

- m: number of nonconstant features; λ: penalty strength.
- **Ridge** (L2) penalizes large coefficients.
- The intercept w_0 is not penalized here.
- Feature scale matters: fit scaling on training data; choose λ on validation data.



Reference: An Additional Regression Metric

On an evaluation set with mean target $\overline{y}_{\text{ev}}$:

$$R^2 = 1 - \frac{\sum_i (y_i - \widehat{y}_i)^2}{\sum_i (y_i - \overline{y}_{\text{ev}})^2}.$$

- $R^2 = 1$ is perfect; 0 matches the evaluation-set mean; negative is worse.
- The formula requires nonconstant targets.
- The evaluation-set mean is a retrospective reference, not an available prediction for future examples.

Key idea

Also compare with a deployable baseline fitted on *training* data, such as its mean target.

A Reproducible Experiment: Split Before Fitting

Predict the diabetes benchmark's numerical one-year progression score using one BMI feature.

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_squared_error, r2_score

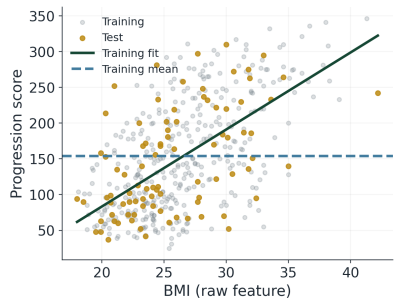
data = load_diabetes(scaled=False)  # raw features
X = data.data[:, [2]]               # BMI: 2-D input
y = data.target
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

This illustrative fixed-model experiment uses 353 training and 89 test observations. No model selection is done on the test set.

Fit on Training Data; Compare on Test Data

```
model = LinearRegression().fit(X_tr, y_tr)
base = DummyRegressor(strategy="mean")
base.fit(X_tr, y_tr)
pred = model.predict(X_te)
base_pred = base.predict(X_te)

print(mean_squared_error(y_te, pred))
print(mean_squared_error(y_te, base_pred))
print(r2_score(y_te, pred))
```



Expected output (rounded)

Linear MSE: **4061.83**
Mean-baseline MSE: **5361.53**
Linear R^2: **0.233**

Modest improvement over a baseline is evidence for this split, not a causal effect or a guarantee about other populations.